

Hoja de ruta del día

- **Módulo:** Módulo III: Machine Learning
- **Semana / Sesión:** Semana 17 - Fase 4: Aprendizaje no supervisado: Clustering y reducción de dimensionalidad
- **Temática Central:** Optimización de Hiper Parámetros: Randomized Search y métricas avanzadas (ROC-AUC, F1-Score).
- **Stack Requerido:** Python, Scikit-Learn, SciPy (stats), Pandas, Matplotlib/Seaborn.

1. Objetivos de aprendizaje del día

Al finalizar la jornada de investigación autónoma, deberás ser capaz de:

- **Objetivo 1:** Comprender e implementar la lógica probabilística y la eficiencia computacional detrás de RandomizedSearchCV frente a GridSearchCV, entendiendo cómo el muestreo aleatorio en un espacio continuo o discreto converge a soluciones óptimas.
- **Objetivo 2:** Identificar los límites y fallos de la implementación estándar de hiper parámetros cuando se trabaja con datasets altamente desbalanceados, y cómo configurar correctamente la estrategia de validación cruzada (StratifiedKFold) para no sesgar las métricas avanzadas.
- **Objetivo 3:** Evaluar y contrastar el impacto real de las métricas F1-Score (balance entre precisión y recall) y ROC-AUC (capacidad de discriminación del modelo) en la toma de decisiones del negocio, seleccionando el mejor set de hiperparámetros en función de la métrica objetivo.

2. Bibliografía

Deberás investigar de forma independiente las siguientes fuentes de referencia para solucionar el reto técnico.

- **Documentación Oficial 1:** [Scikit-Learn API — RandomizedSearchCV documentation](#) (parámetros param_distributions, n_iter, scoring, refit).
- **Documentación Oficial 2:** [Scikit-Learn User Guide — Tuning the hyper-parameters of an estimator](#) (comparativa Grid vs Randomized Search).
- **Documentación Oficial 3:** [Scikit-Learn API](#) métricas de clasificación— (roc_auc_score, f1_score, classification_report y precision_recall_curve.)

- **Documentación Oficial 4:** Scikit-Learn API: [Tuning the hyper-parameters of an estimator \(Randomized Parameter Optimization\)](#).
- **Documentación Oficial 5:** Scikit-Learn Metrics API: [Classification metrics \(ROC-AUC and F1-Score definition & multi-class trade-offs\)](#).

3. Ejemplo de implementación práctica

Breve fragmento de código que ilustra el patrón de diseño correcto sin resolver el problema central.

```
Python
from sklearn.model_selection import RandomizedSearchCV

from scipy.stats import randint, uniform

from sklearn.ensemble import RandomForestClassifier

# Definición del espacio de búsqueda como DISTRIBUCIONES, no listas fijas

param_dist = {

    "n_estimators": randint(50, 300),

    "max_depth": randint(3, 20),

    "min_samples_split": uniform(0.01, 0.2)

}

modelo = RandomForestClassifier(random_state=42)

busqueda = RandomizedSearchCV(

    estimator=modelo,

    param_distributions=param_dist,

    n_iter=25,                # n° de combinaciones muestreadas, no todas

    scoring="roc_auc",        # métrica de referencia para elegir el mejor modelo

    cv=5,

    random_state=42,

    n_jobs=-1

)

# busqueda.fit(X_train, y_train) -> el ajuste sobre el dataset del reto
```

Este ejemplo muestra cómo se declara una búsqueda aleatoria, pero no resuelve el problema de optimización multi-métrica ni la elección de scoring que plantea el reto.

4. Reto práctico de código:

Dataset del reto:  [Credit Card Fraud Detection](#) (ULB — Machine Learning Group, Université Libre de Bruxelles). Transacciones europeas de tarjeta de crédito de septiembre de 2013, con variables anonimizadas mediante PCA, más *Time*, *Amount* y la variable objetivo *Class*.

Contexto del problema: Una entidad financiera está desarrollando un modelo de **detección de fraude en transacciones con tarjeta**. El dataset histórico tiene un fuerte desbalanceo de clases: solo el 2% de las transacciones son fraudulentas. El equipo de riesgo necesita un modelo afinado que minimice tanto el coste de dejar pasar fraudes (falsos negativos) como el de bloquear transacciones legítimas (falsos positivos), sabiendo que ambos errores tienen un coste de negocio distinto.

Componente de incertidumbre: `RandomizedSearchCV` permite optimizar sobre **una sola métrica** a través del parámetro `scoring`, pero el equipo de riesgo pide comparar simultáneamente el comportamiento del modelo en ROC-AUC y F1-Score antes de decidir cuál usar como criterio de selección final. Deberás investigar:

- Cómo usar `scoring` con múltiples métricas (`scoring=["roc_auc", "f1"]`) junto con el parámetro `refit`, y qué implica que `refit` deba apuntar a una sola métrica para poder usar `.predict()` después.
- Cómo interpretar la matriz de resultados (`cv_results_`) cuando se optimizan varias métricas a la vez, sin haber sido presentado explícitamente este caso en sesiones anteriores.
- Por qué, con un desbalanceo del 2%, el *accuracy* no debe usarse como criterio de decisión y qué relación existe entre el umbral de clasificación (*threshold*) y el equilibrio entre Precision y Recall reflejado en el F1-Score.

Restricción ética y responsabilidad: Antes de entregar el modelo final, deberás responder por escrito, dentro del notebook, a la siguiente pregunta:

Si el modelo optimizado prioriza maximizar el Recall (para no dejar pasar fraudes) a costa de reducir la Precision (más transacciones legítimas bloqueadas), ¿qué impacto tiene esta decisión sobre usuarios reales, y cómo debería comunicarse esa asimetría de errores a un equipo no técnico que debe aprobar el despliegue del modelo?

La respuesta debe evidenciar comprensión del *trade-off* Precision/Recall, no solo su cálculo matemático.